

SPT | Frame | Swap

Lazy Load | Stack Growth

Clock Eviction | mmap

113 / 113 testes

Relatorio de Decisoões de Projeto

Projeto 3 — Virtual Memory (Conclusao)

Centro de Informatica — Universidade Federal de Pernambuco

Disciplina: Sistemas Operacionais

2025 / 2026

1. Sumario Executivo

Este relatório documenta a conclusão do **Projeto 3 de Memória Virtual** do PintOS, construída sobre a Frame Table entregue anteriormente. Ao final desta etapa, o kernel possui suporte completo a memória virtual sob demanda: tabela de páginas suplementar (SPT) por processo, algoritmo de relógio para evicção de frames, swap em disco, crescimento dinâmico da pilha, carregamento lento de executáveis e arquivos mapeados na memória (**mmap/munmap**).

Resultado dos testes: 113 / 113 (100%)

Arquivo	Status	Função
vm/page.h	Novo	Define sup_page_entry, mmap_entry e prototipos do SPT.
vm/page.c	Novo	Implementa SPT (hash por processo), spt_load_page, munmap_entry, spt_destroy.
vm/swap.h	Novo	Define swap_init, swap_write, swap_read, swap_free.
vm/swap.c	Novo	Implementa bitmap de slots e I/O com BLOCK_SWAP.
vm/frame.h	Modificado	Adiciona campos spte e pinned; novos prototipos frame_remove, frame_unpin.
vm/frame.c	Modificado	Adiciona algoritmo de relógio, evicção, pinning, frame_remove, frame_unpin.
userprog/exception.c	Modificado	Handler de page fault: lazy load, stack growth, recuperação em modo kernel.
userprog/process.c	Modificado	load_segment com SPT lazy, setup_stack via SPT, spt_destroy em process_exit.
userprog/syscall.c	Modificado	SYS_MMAP, SYS_MUNMAP, esp_saved, put_user em validate_buffer.
threads/thread.h	Modificado	Campos VM: spt_initialized, spage_table, mmap_list, next_mapid, esp_saved.
threads/init.c	Modificado	Chama frame_init() e swap_init() após filesystem_init() sob #ifdef VM.
Makefile.build	Modificado	vm_SRC com page.c e swap.c; correção do loader.bin para GNU ld 2.45.

2. Arquitetura do Subsistema de Memória Virtual

O subsistema é organizado em três camadas sobrepostas que se acionam sequencialmente a cada page fault:

Processo usuário

|

```
v page fault
exception.c <-- identifica causa: stack growth / lazy load / swap reclaim
|
v
vm/page.c <-- SPT: o que deve estar nessa pagina virtual?
|
v
vm/frame.c <-- frame table: aloca frame fisico; evicta via clock se necessario
|
v
vm/swap.c <-- swap: leitura/escrita de paginas no disco
```

Invariante central: toda pagina virtual de usuario e descrita por uma entrada **sup_page_entry** no SPT da thread. O frame fisico e alocado sob demanda, na primeira falha de pagina. Antes disso, a pagina existe apenas como metadados no SPT.

3. vm/page.h — Estruturas de Dados

3.1 enum page_location

vm/page.h — enum page_location

```
enum page_location {
    PAGE_ZERO, /* zeros puros (stack growth ou BSS) */
    PAGE_FILE, /* arquivo (executavel ou mmap) */
    PAGE_SWAP, /* expulso para swap */
    PAGE_FRAME, /* atualmente em frame fisico */
};
```

O enum descreve onde estão os dados de uma página virtual em cada momento. O handler de page fault e **spt_load_page** usam esse campo para decidir de onde carregar os dados: do arquivo, do swap, ou simplesmente zerando o frame.

3.2 struct sup_page_entry

vm/page.h — struct sup_page_entry

```
struct sup_page_entry {
    struct hash_elem hash_elem;
    void *upage; /* endereço virtual (alinhado a página) – chave do hash */
    enum page_location location;
    bool writable;
    bool dirty; /* acumulado para write-back durante evicão */
    bool is_mmap; /* página pertence a um mmap? */
    int mapid;
    struct file *file;
    off_t file_offset;
    size_t read_bytes;
    size_t zero_bytes;
    size_t swap_sector; /* setor inicial no dispositivo de swap */
    void *kpage; /* endereço kernel quando PAGE_FRAME */
};
```

Campo	Finalidade
upage	Chave do hash. Identifica univocamente a página virtual no espaço do processo.
location	Diz onde estão os dados: RAM, arquivo, swap ou zero puro.
dirty	Preserva o bit dirty entre evicões. Essencial para write-back correto de páginas mmap evictadas para swap.
is_mmap	Distingue páginas mmap (write-back para arquivo) de páginas normais (vão para swap).
swap_sector	Índice do setor inicial no bitmap de swap quando location == PAGE_SWAP.
kpage	Endereço kernel válido somente quando location == PAGE_FRAME.

3.3 struct mmap_entry

vm/page.h — struct mmap_entry

```
struct mmap_entry {
```

```

    int mapid;
    struct file *file; /* referencia independente via file_reopen */
    void *addr; /* inicio da regioa mapeada */
    size_t page_count;
    struct list_elem elem;
};

```

Uma entrada por chamada bem-sucedida a **mmap**. O campo **file** é uma referência independente aberta com **file_reopen** — o descritor de arquivo original pode ser fechado a qualquer momento sem afetar o mapeamento.

4. vm/page.c — Implementação do SPT

4.1 Hash table por processo

O SPT usa **struct hash** do PintOS indexada por **upage**. A busca (**spt_find**) é $O(1)$ esperado — essencial pois ocorre em todo page fault. Como o SPT é privado por thread, não é necessário lock para leituras.

vm/page.c — funções de hash

```

static unsigned
page_hash(const struct hash_elem *e, void *aux UNUSED) {
    const struct sup_page_entry *spte =
        hash_entry(e, struct sup_page_entry, hash_elem);
    return hash_bytes(&spte->upage, sizeof spte->upage);
}

static bool
page_less(const struct hash_elem *a,
           const struct hash_elem *b, void *aux UNUSED) {
    return hash_entry(a, struct sup_page_entry, hash_elem)->upage
        < hash_entry(b, struct sup_page_entry, hash_elem)->upage;
}

```

4.2 spt_load_page() — carregamento sob demanda

Chamada pelo handler de page fault após localizar a entrada SPT. Aloca um frame (com pinning), preenche de acordo com a localização dos dados, instala o mapeamento no pagedir e desafina o frame.

vm/page.c — spt_load_page()

```

bool spt_load_page(struct sup_page_entry *spte) {
    void *kpage = frame_alloc(spte,
                              PAL_USER | (spte->location == PAGE_ZERO ? PAL_ZERO : 0));
    if (!kpage) return false;
    switch (spte->location) {
    case PAGE_ZERO:
        memset(kpage, 0, PGSIZE);
        break;
    case PAGE_FILE:
        file_read_at(spte->file, kpage, spte->read_bytes, spte->file_offset);
        memset(kpage + spte->read_bytes, 0, spte->zero_bytes);
        break;
    case PAGE_SWAP:
        swap_read(spte->swap_sector, kpage);
        swap_free(spte->swap_sector);
    }
}

```

```

        break;
    default:
        frame_free(kpage);
        return false;
    }
    pagedir_set_page(thread_current()->pagedir,
                    spte->upage, kpage, spte->writable);
    spte->kpage = kpage;
    spte->location = PAGE_FRAME;
    frame_unpin(kpage); /* permite evicao apos mapeamento instalado */
    return true;
}

```

Atencao: `file_read_at` e chamado SEM `filesys_lock`. Adquiri-lo causaria deadlock: `SYS_READ` pode estar segurando o lock quando o page fault ocorre dentro de `file_read`. O executavel tem `file_deny_write` ativo, tornando leituras concorrentes seguras.

4.3 munmap_entry() — write-back e liberacao

Percorre cada pagina do mapeamento e decide o que fazer com base no estado atual da pagina. A logica de dirty e combinada: **`pagedir_is_dirty` OR `spte->dirty`**. O campo `spte->dirty` e essencial porque quando uma pagina mmap e evictada, **`pagedir_clear_page`** apaga o bit dirty do pagedir — o campo acumula esse estado para que o write-back seja correto.

Estado da pagina	Acao em munmap_entry
PAGE_FRAME, dirty	<code>file_write_at</code> → <code>pagedir_clear_page</code> → <code>frame_free</code>
PAGE_FRAME, limpa	<code>pagedir_clear_page</code> → <code>frame_free</code> (sem write-back)
PAGE_SWAP, dirty	<code>swap_read</code> para buffer temporario → <code>file_write_at</code> → <code>swap_free</code>
PAGE_SWAP, limpa	<code>swap_free</code> apenas (dado original no arquivo ainda valido)
PAGE_FILE / PAGE_ZERO	Nenhuma acao de I/O (nunca foi acessada)

4.4 spt_destroy() — destruicao em process_exit

O destrutor chamado por **`hash_destroy`** para cada entrada decide a acao com base em **`location`**:

location	Acao
PAGE_FRAME	<code>frame_remove(kpage)</code> — remove da tabela SEM <code>palloc_free</code> . <code>pagedir_destroy</code> libera o frame fisico.
PAGE_SWAP	<code>swap_free(swap_sector)</code> — libera o slot no bitmap.
PAGE_FILE / PAGE_ZERO	Nenhuma acao de I/O necessaria.

Por que `frame_remove` e nao `frame_free`? `pagedir_destroy`, chamado logo apos `spt_destroy`, percorre todas as PTEs com `PTE_P=1` e devolve os frames ao palloc. Chamar `palloc_free_page` no destrutor do SPT causaria double-free.

5. vm/swap.h + vm/swap.c

5.1 Estado global

vm/swap.c — variáveis estáticas

```
static struct block *swap_block;
static struct bitmap *swap_bitmap; /* 1 bit por slot; 1 slot = 1 página */
static struct lock swap_lock;
#define SECTORS_PER_PAGE (PGSIZE / BLOCK_SECTOR_SIZE) /* = 8 */
```

swap_init obtém o dispositivo **BLOCK_SWAP**, calcula o número de slots (**block_size / SECTORS_PER_PAGE**) e aloca o bitmap com **bitmap_create**. Cada bit representa um slot (uma página = 8 setores).

5.2 swap_write() e swap_read()

vm/swap.c — swap_write()

```
size_t swap_write(void *kpage) {
    lock_acquire(&swap_lock);
    size_t slot = bitmap_scan_and_flip(swap_bitmap, 0, 1, false);
    lock_release(&swap_lock);
    if (slot == BITMAP_ERROR)
        PANIC("swap: no free slot");
    size_t base = slot * SECTORS_PER_PAGE;
    for (size_t i = 0; i < SECTORS_PER_PAGE; i++)
        block_write(swap_block, base + i,
                    (uint8_t *)kpage + i * BLOCK_SECTOR_SIZE);
    return base; /* setor inicial -- armazenado em spte->swap_sector */
}
```

vm/swap.c — swap_read()

```
void swap_read(size_t sector, void *kpage) {
    for (size_t i = 0; i < SECTORS_PER_PAGE; i++)
        block_read(swap_block, sector + i,
                    (uint8_t *)kpage + i * BLOCK_SECTOR_SIZE);
}
```

Por que o lock só cobre **bitmap_scan_and_flip** e não o I/O? **block_write** e **block_read** já são thread-safe internamente. Manter **swap_lock** durante o I/O de disco bloquearia desnecessariamente outras threads que precisam apenas de slots diferentes.

6. vm/frame.c — Extensões para Evicão

6.1 Novos campos em struct frame_entry

vm/frame.h — frame_entry (versão final)

```
struct frame_entry {
    struct list_elem elem;
    void *kpage;
    struct thread *owner;
    struct sup_page_entry *spte; /* NOVO: entrada SPT correspondente */
    bool pinned; /* NOVO: true = não elegível para evicão */
};
```


spte substitui o antigo campo **upage** — o frame agora conhece toda a descrição da página, não só o endereço virtual. Isso permite que a evicção acesse **file**, **swap_sector**, **is_mmap** e **read_bytes** diretamente. **pinned** garante que um frame em processo de população não seja evictado por outra thread antes do mapeamento estar instalado.

6.2 frame_alloc() — alocação com evicção

vm/frame.c — frame_alloc()

```
void *frame_alloc(struct sup_page_entry *spte, enum palloc_flags flags) {
    lock_acquire(&frame_lock);
    void *kpage = palloc_get_page(flags);
    if (kpage != NULL) {
        struct frame_entry *fte = malloc(sizeof *fte);
        fte->kpage = kpage;
        fte->owner = thread_current();
        fte->spte = spte;
        fte->pinned = true; /* pinado ate frame_unpin apos pagedir_set_page */
        list_push_back(&frame_table, &fte->elem);
        lock_release(&frame_lock);
        return kpage;
    }
    /* Memoria fisica esgotada - evicta. */
    kpage = frame_evict(spte, flags);
    lock_release(&frame_lock);
    return kpage;
}
```

Decisão	Motivo
pinned=true na criação	O frame ainda não tem mapeamento no pagedir. Evicta-lo antes causaria inconsistência entre frame_table e pagedir.
frame_unpin após pagedir_set_page	Só após o mapeamento estar instalado o algoritmo de relógio pode acessar os bits de acesso do pagedir.
spte no lugar de upage	Evicção precisa de todos os metadados da página para decidir se vai para swap ou arquivo.

6.3 Algoritmo de Relógio (Clock / Second-Chance)

Quando **palloc_get_page** retorna NULL, **frame_evict** percorre a lista circular de frames com um ponteiro **clock_hand**. Frames pinados são ignorados. Frames com bit de acesso ativo recebem **segunda chance** (bit zerado, ponteiro avança). O primeiro frame com bit de acesso zero é selecionado como vítima.

vm/frame.c — frame_evict() (resumido)

```
/* Vítima selecionada. */
bool dirty = pagedir_is_dirty(fte->owner->pagedir, fte->spte->upage)
    || fte->spte->dirty;
pagedir_clear_page(fte->owner->pagedir, old_spte->upage);
old_spte->kpage = NULL;
old_spte->dirty = dirty;
/* Atualiza frame_entry in-place antes de soltar o lock. */
fte->spte = new_spte;
fte->owner = thread_current();
```

```

fte->pinned = true;
lock_release(&frame_lock);
/* I/O sem frame_lock para evitar deadlock com filesystem_lock. */
if (old_spte->is_mmap) {
    if (dirty) { file_write_at(...); }
    old_spte->location = PAGE_FILE;
} else {
    old_spte->swap_sector = swap_write(kpage);
    old_spte->location = PAGE_SWAP;
}

```

Atencao: *frame_lock* e solto ANTES do I/O de disco. Manter o lock durante *swap_write* ou *file_write_at* causaria deadlock: outra thread em page fault chamaria *frame_alloc*, tentaria adquirir *frame_lock* e ficaria presa enquanto a primeira espera o disco. O frame e atualizado in-place antes de soltar o lock para que nenhum outro thread o selecione como vitima.

6.4 frame_remove vs frame_free

Funcao	Remove da tabela	palloc_free_page	Uso
frame_free	Sim	Sim	munmap_entry, erros em spt_load_page.
frame_remove	Sim	Nao	Destrutor SPT — pagedir_destroy libera o frame fisico.

7. userprog/exception.c — Handler de Page Fault

7.1 Fluxo em page_fault()

O handler percorre tres caminhos possiveis em ordem:

Passo	Condicao	Acao
1 — Recuperaao kernel	!user && f->eax != 0 (label de recuperacao em %eax)	f->eip = f->eax; f->eax = -1; return. Permite get_user/put_user falharem sem crash.
2 — Lazy load / swap reclaim	spt_find encontra spte para o upage faltante	spt_load_page(spte): aloca frame, preenche dados, instala mapeamento.
3 — Stack growth	Endereco dentro dos limites validos de crescimento	Cria nova spte (PAGE_ZERO), insere no SPT, chama spt_load_page.

7.2 Condicoes de stack growth

userprog/exception.c — condicoes de stack growth

```
void *user_esp = user ? f->esp : cur->esp_saved;
if (user_esp != NULL
    && (uintptr_t)fault_addr < (uintptr_t)PHYS_BASE
    && (uintptr_t)fault_addr >= (uintptr_t)PHYS_BASE - 8 * 1024 * 1024
    && (uintptr_t)fault_addr >= (uintptr_t)user_esp - 32)
{ /* stack growth */ }
```

Condicao	Razao
fault_addr < PHYS_BASE	Endereco de usuario valido.
>= PHYS_BASE - 8 MB	Pilha limitada a 8 MB pelo enunciado do projeto.
>= user_esp - 32	Cobre a instrucao PUSHBA que empurra 32 bytes abaixo de esp antes de atualiza-lo.

7.3 esp_saved — ESP do usuario em contexto de kernel

Quando o page fault ocorre dentro de uma syscall (modo kernel), **f->esp** aponta para a pilha do kernel — nao a pilha do usuario. O ESP real e salvo em **thread->esp_saved** no inicio do **syscall_handler** e limpo ao sair.

userprog/syscall.c — salvando e restaurando esp_saved

```
void syscall_handler(struct intr_frame *f) {
    thread_current()->esp_saved = f->esp;
    /* ... despacho das syscalls ... */
    thread_current()->esp_saved = NULL;
}
```

userprog/exception.c — usando esp_saved

```
void *user_esp = user ? f->esp : cur->esp_saved;
```

8. userprog/process.c — Lazy Loading

8.1 load_segment() com SPT

Antes: alocava um frame e lia o arquivo imediatamente para cada página. Agora: cria apenas entradas SPT sem alocar nenhum frame. Os frames são alocados sob demanda no primeiro acesso a cada página.

userprog/process.c — load_segment() com VM (trecho central)

```
struct sup_page_entry *spte = malloc(sizeof *spte);
spte->upage = upage;
spte->location = (page_read_bytes == 0) ? PAGE_ZERO : PAGE_FILE;
spte->file = file;
spte->file_offset = file_offset;
spte->read_bytes = page_read_bytes;
spte->zero_bytes = page_zero_bytes;
spte->writable = writable;
spte->is_mmap = false;
spte->dirty = false;
spt_insert(&thread_current()->spte_table, spte);
```

8.2 setup_stack() via SPT

A pilha inicial é tratada como qualquer outra página — criada no SPT e carregada via **spt_load_page**. Ao contrário das páginas do executável, ela é carregada imediatamente porque precisa existir antes de executar qualquer instrução.

userprog/process.c — setup_stack() via SPT

```
struct sup_page_entry *spte = malloc(sizeof *spte);
spte->upage = ((uint8_t *)PHYS_BASE) - PGSIZE;
spte->location = PAGE_ZERO;
spte->writable = true;
spte->is_mmap = false;
spte->dirty = false;
spt_insert(&thread_current()->spte_table, spte);
if (!spt_load_page(spte)) return false;
*esp = PHYS_BASE;
return true;
```

8.3 Ordem de liberação em process_exit()

userprog/process.c — process_exit() (ordem crítica)

```
/* 1. Fecha fds e executável. */
/* 2. Write-back de todas as páginas mmap sujas. */
munmap_all(cur);
/* 3. Destroi SPT (libera slots de swap, remove entradas do frame table). */
if (cur->spt_initialized)
    spt_destroy(&cur->spte_table);
/* 4. Sinaliza pai, libera filhos. */
/* 5. Destroi page directory (libera frames físicos com PTE_P=1). */
pagedir_destroy(pd);
```

Por que essa ordem? `munmap_all` precisa do `pagedir` (para verificar `dirty`) e de `spte->kpage` (para copiar os dados). `spt_destroy` usa `frame_remove` sem `palloc_free`. `pagedir_destroy` libera os frames físicos — por isso o destrutor `SPT` não chama `palloc_free_page`. Inverter as etapas 2 e 3 causaria acesso a `kpage` inválido; inverter 3 e 5 causaria `double-free`.

9. userprog/syscall.c — mmap, munmap e put_user

9.1 sys_mmap() — validacao e criacao lazy

Valida o endereco (nao NULL, alinhado a pagina, fd valido, arquivo nao vazio), verifica que nenhuma pagina sobrepoa mapeamento existente no SPT, abre referencia independente com **file_reopen** e cria entradas SPT do tipo **PAGE_FILE** com **is_mmap=true** para cada pagina. As paginas sao carregadas lazily — somente no primeiro acesso.

userprog/syscall.c — sys_mmap() (estrutura)

```
/* file_reopen: referencia independente do fd */
struct file *mmap_file = file_reopen(orig);
/* Verifica sobreposicao */
for (size_t i = 0; i < page_count; i++)
    if (spt_find(&cur->sptable, addr + i * PGSIZE) != NULL)
        return -1; /* sobreposicao detectada */
/* Cria mmap_entry e entradas SPT */
for (size_t i = 0; i < page_count; i++) {
    struct sup_page_entry *spte = malloc(sizeof *spte);
    spte->upage = addr + i * PGSIZE;
    spte->location = PAGE_FILE;
    spte->is_mmap = true;
    spte->mapid = me->mapid;
    spte->file = mmap_file;
    spte->file_offset = i * PGSIZE;
    /* ... read_bytes, zero_bytes, writable ... */
    spt_insert(&cur->sptable, spte);
}
return me->mapid;
```

Por que file_reopen? O fd do processo pode ser fechado a qualquer momento apos mmap. file_reopen cria uma referencia independente para o mapeamento, garantindo que o arquivo permaneça acessível durante toda a vida do mmap — mesmo apos close(fd).

9.2 sys_munmap()

munmap_all percorre **mmap_list** e chama **munmap_entry** para cada entrada. É chamado em **process_exit** para garantir write-back mesmo que o processo termine sem chamar munmap explicitamente.

9.3 put_user — validacao de escrita antes do lock

userprog/syscall.c — put_user()

```
static bool put_user(uint8_t *udst, uint8_t byte) {
    if (!is_user_vaddr(udst)) return false;
    int error_code;
    asm ("movl $1f, %0; movb %b2, %1; 1:"
        : "=a" (error_code), "=m" (*udst) : "q" (byte));
    return error_code != (int)0xffffffff;
}
```

validate_buffer(..., writable=true) chama **put_user** para verificar permissao de escrita **antes** de adquirir **filesystem_lock**. Isso resolve o teste **pt-write-code2**:

Sem put_user	Com put_user
validate_buffer carrega a pagina read-only.	put_user tenta escrever na pagina read-only.
lock_acquire(&fileys;_lock) adquirido.	Page fault na instrucao de escrita da assembly.
file_read tenta escrever na pagina → page fault em modo kernel.	Handler encontra %eax com label de recuperacao.
Sem label de recuperacao → deadlock / timeout.	put_user retorna false → exit(-1) antes do lock.

10. threads/thread.h — Campos VM

threads/thread.h — campos VM em struct thread

```
#ifdef VM
bool spt_initialized; /* true apos spt_init() em start_process */
struct hash spage_table; /* tabela de paginas suplementar */
struct list mmap_list; /* entradas de mmap ativas */
int next_mapid; /* proximo ID de mmap a atribuir */
void *esp_saved; /* ESP do usuario salvo no syscall_handler */
#endif
```

Todos são inicializados em **start_process** antes de qualquer **thread_exit()**. O campo **spt_initialized**, inicializado como **false** pelo **pallocc** com **PAL_ZERO**, garante que threads de kernel e processos que falham antes de **spt_init** não tenham **spt_destroy** chamado sobre estruturas inválidas.

11. Inicializacao e Build

11.1 threads/init.c

threads/init.c — inicializacao do subsistema VM

```
#ifdef VM
frame_init(); /* inicializa lista e lock da frame table */
swap_init(); /* obtem BLOCK_SWAP e cria bitmap */
#endif
```

Chamadas após **fileys_init()** e antes do primeiro processo de usuário. A ordem é obrigatória: **swap_init** precisa que o sistema de blocos esteja pronto.

11.2 Makefile.build — vm_SRC e correcao do loader.bin

Makefile.build — ativacao dos tres modulos VM

```
vm_SRC = vm/frame.c
vm_SRC += vm/page.c
vm_SRC += vm/swap.c
```

Makefile.build — correcao para GNU ld 2.45 (Fedora 43)

```
loader.bin: threads/loader.o
$(OBJCOPY) --remove-section=.note.gnu.property $< threads/loader_stripped.o
$(LD) -N -e 0 -Ttext 0x7c00 --format binary -o $@ threads/loader_stripped.o
rm -f threads/loader_stripped.o
```

O GNU ld 2.45 insere automaticamente uma seção **.note.gnu.property** com LMA **0x080480b4**, fazendo o binário bruto ter 134 MB em vez de 512 bytes. **objcopy --remove-section** remove a seção antes da linkagem, produzindo um **loader.bin** de exatamente 512 bytes, compatível com o script **pintos**.

12. Decisões de Projeto

Decisão	Alternativa Descartada	Justificativa
Hash table no SPT indexada por upage	Lista encadeada ou array	$O(1)$ esperado para lookup no page fault. Um processo pode ter centenas de páginas.
spt_initialized como flag booleana	Verificar se spage_table.bucket e NULL	palloc com PAL_ZERO garante false como estado inicial sem inicialização extra.
file_read_at sem filesys_lock em spt_load_page	Adquirir filesys_lock antes da leitura	Deadlock inevitável: SYS_READ pode estar com o lock quando o page fault ocorre.
Frame atualizado in-place antes de soltar frame_lock	Alocar novo frame_entry para o novo dono	Evita janela de corrida onde outro thread seleciona o mesmo frame como vítima.
pinned=true até frame_unpin após pagedir_set_page	Não pinar frames	Sem pinning, evicão poderia selecionar frame que ainda não tem mapeamento instalado.
munmap_all antes de spt_destroy em process_exit	Destruir SPT e depois fazer write-back	munmap_entry precisa do pagedir para verificar dirty e de spte->kpage para copiar.
frame_remove sem palloc_free no destrutor SPT	Chamar palloc_free_page no destrutor	pagedir_destroy já libera frames com PTE_P=1; palloc_free_page causaria double-free.
esp_saved no struct thread	Passar ESP como argumento para o handler	Handler de page fault não tem acesso ao frame de interrupção da syscall.
put_user antes de filesys_lock em SYS_READ	Verificar permissão dentro do lock	Page fault em página read-only com lock adquirido causaria deadlock.
file_reopen para mmap	Reutilizar o ponteiro do fd	O fd pode ser fechado após mmap; a referência deve ser independente.
Bitmap para slots de swap	Lista de slots livres	bitmap_scan_and_flip é $O(n/\text{word})$ e atômica; lista exigiria lock e malloc separados.

13. Invariantes do Sistema

Invariante	Como é Garantido
Toda página virtual de usuário tem entrada SPT	load_segment, setup_stack e stack growth criam entradas antes de qualquer acesso.

Frame físico nunca é double-freed	<code>frame_remove</code> (sem <code>palloc_free</code>) no destrutor SPT; <code>pagedir_destroy</code> libera os frames.
Página mmap suja sempre escrita de volta	<code>munmap_entry</code> verifica <code>pagedir_is_dirty</code> OR <code>spte->dirty</code> .
Evicção não causa deadlock com <code>fileysys_lock</code>	<code>frame_lock</code> é solto antes de qualquer I/O de arquivo ou swap.
Frame pinado não é evictado	<code>pinned=true</code> até <code>frame_unpin</code> após <code>pagedir_set_page</code> .
Kernel nunca crasha por ponteiro inválido de usuário	<code>get_user/put_user</code> com recuperação assembly; page fault em modo kernel usa label em <code>%eax</code> .
Pilha não cresce além de 8 MB	Condição <code>fault_addr >= PHYS_BASE - 8MB</code> verificada em cada stack growth.
Swap não estoura silenciosamente	PANIC('swap: no free slot') se <code>bitmap_scan_and_flip</code> retorna <code>BITMAP_ERROR</code> .
<code>SYS_READ</code> em página read-only encerra antes do lock	<code>put_user</code> em <code>validate_buffer</code> detecta falta de permissão antes de <code>fileysys_lock</code> .
Threads de kernel não chamam <code>spt_destroy</code>	Campo <code>spt_initialized</code> (false por default via <code>PAL_ZERO</code>) protege <code>process_exit</code> .